

AI RESEARCH ASSISTANT

Project Report

Name: Arpit Abhay Kulkarni, Vedant Yogesh Yadao, Batul Shabbir Jabalpurwala

Domain: AI/ML

College: Government Polytechnic College, Washim

1. Abstract

The AI Research Assistant is a document-based question-answering system designed to help users work with research papers and other documents.

It accepts PDF, DOCX, TXT, and PPTX files through a Streamlit interface, extracts and cleans their text, divides content into chunks, converts chunks into embeddings, retrieves semantically relevant chunks for a question, and uses a large language model to generate a grounded response.

The system also supports document summaries and source information.

2. Problem Statement

Long research documents are difficult to search manually.

Users may need to locate findings, understand methodologies, compare papers, or obtain summaries quickly.

The system provides a natural-language interface over uploaded documents and instructs the language model to rely on supplied research context rather than unsupported assumptions.

3. Objectives

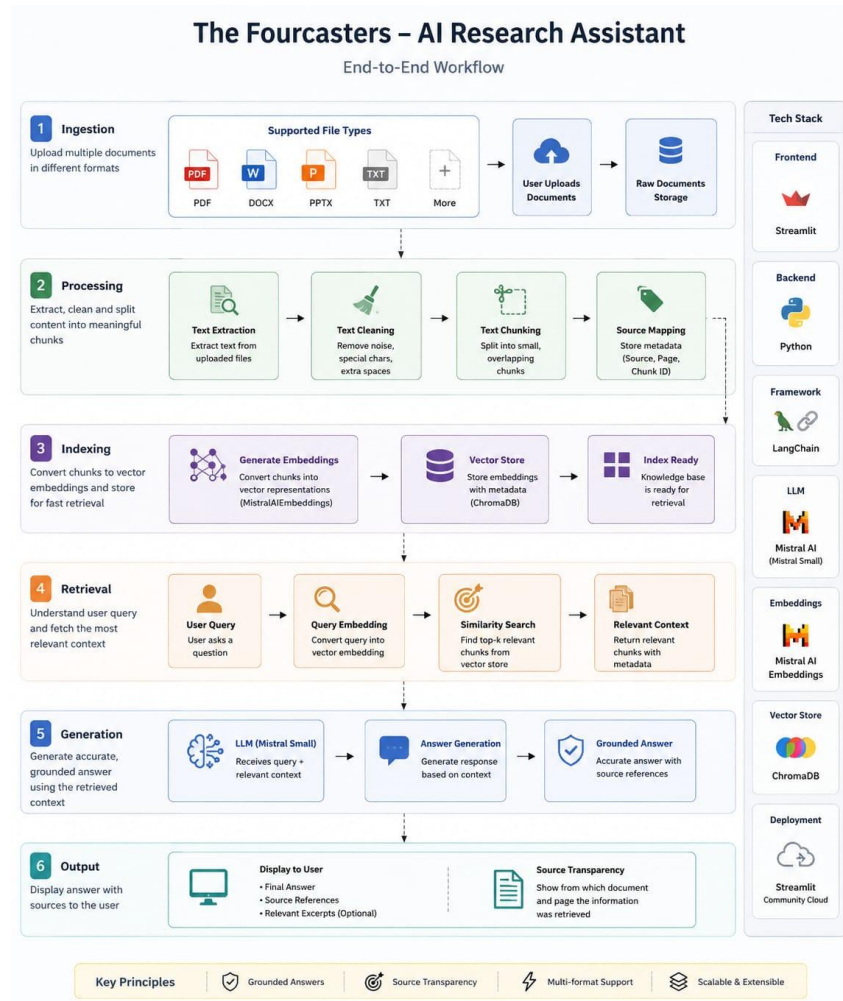
- Support multiple uploaded research documents.
- Support PDF, DOCX, TXT, and PPTX formats.
- Extract, clean, and chunk document text while retaining location metadata.
- Generate embeddings for document chunks and user questions.

- Retrieve relevant chunks using cosine similarity.
- Generate source-grounded answers using an LLM.
- Generate concise document summaries.
- Provide an interactive Streamlit interface.

4. System Architecture

- User uploads documents through Streamlit.
- Uploaded files are temporarily stored for backend processing.
- The processing module detects the file type and extracts text.
- Extracted text is cleaned and divided into overlapping chunks.
- The backend generates embeddings for the chunks.
- The question is embedded and compared with document embeddings.
- The most relevant chunks are selected.
- The retrieved context is sent to the Mistral language model.
- The generated answer and source information are shown in the UI.

5. System Workflow



6. Data Processing

The Dataprocessing.py module converts different document formats into a common structure for retrieval.

6.1 PDF Processing

PDF documents are processed using PyMuPDF. The processor reads the document page by page, extracts textual content, and retains page numbers.

6.2 DOCX Processing

DOCX files are processed using python-docx. The system reads document paragraphs and retains paragraph numbers as metadata.

6.3 TXT Processing

TXT files are read using standard Python file handling with UTF-8 encoding and separated into paragraphs.

6.4 PPTX Processing

PPTX files are processed using python-pptx. Text is extracted from slide shapes and slide numbers are retained.

6.5 Text Cleaning

Extracted text is cleaned to reduce unnecessary spaces, blank lines, and formatting inconsistencies before retrieval.

6.6 Chunking

Large documents are divided into smaller chunks. Overlap between chunks helps preserve context when information spans consecutive chunks.

6.7 Automatic Processing

The process_file() function automatically selects the appropriate processor based on the file extension.

7. Semantic Retrieval

The backend uses MistralAIEmbeddings with the mistral-embed model to convert document chunks into numerical vectors.

The user's question is embedded using the same model.

Cosine similarity is calculated between the question vector and each chunk vector.

The chunks are sorted by similarity and the top five positive-scoring chunks are passed to the answer-generation model.

This approach is more flexible than simple keyword matching because related meaning can be retrieved even when the question and document use different wording.

8. LLM Answer Generation

The backend initializes the Mistral Small chat model through LangChain.

Retrieved chunks are combined with source and page metadata into a research context.

The prompt instructs the model to use only the provided context, avoid inventing information, identify relevant sources, explain conflicts between sources, and compare multiple documents in a table when appropriate.

9. Document Summarization

The summarization function collects chunks belonging to a selected document, orders them using page, paragraph, and slide metadata, limits the input to a maximum word count, and asks the LLM to summarize the document's objective, methodology or findings, and conclusions.

10.1 Streamlit Frontend

The frontend provides multiple-file upload, document processing, loaded-document management, summaries, chat-based questions, and a Sources section.

Uploaded files are written to temporary files, processed, embedded, and stored in Streamlit session state. Questions are answered using semantic retrieval over the stored chunks.

10.2 Output

The screenshot displays the 'AI Research Assistant' web application. On the left sidebar, under 'Upload Documents', a file named 'HACKATHON_101_Complete_First-Time_Participant_Guide.pdf' (496.1KB) is listed. Below it is a 'Process Documents' button. The 'Loaded Documents' section shows the same file with a 'Summarize' button and a 'Clear All Documents' button. The main content area is titled 'AI Research Assistant' and includes instructions to 'Upload PDF, DOCX, TXT, or PPTX files. Ask grounded questions, get summaries, and compare papers.' Below this, the 'Document Summaries' section shows a dropdown menu with the selected file. The summary for 'HACKATHON_101_Complete_First-Time_Participant_Guide.pdf' is displayed, featuring a 'Main Objective' section that states: 'The guide aims to help first-time hackathon participants (students and developers) understand how to effectively plan, build, and present a solution under time pressure. It emphasizes that a hackathon is not about writing the most code but solving a real problem, proving the idea, and communicating its value.' At the bottom of the main area is a text input field with the placeholder 'Ask a question about your documents...' and a submit button.

This screenshot shows the same 'AI Research Assistant' interface after a query has been entered. The 'Upload Documents' and 'Loaded Documents' sections remain unchanged. In the main content area, the 'Document Summaries' section now displays a query in the input field: 'mistakes to avoid?'. Below the input field, a response is shown: 'Here are the common mistakes to avoid during a hackathon, based on the provided research context:'. The response is titled 'Common First-Time Mistakes (Page 19)' and lists three categories of mistakes: 1. Idea too large, 2. Coding before planning, and 3. Unnecessary tech. Each category includes sub-points explaining why it happens, why it hurts, and how to avoid it. For example, under 'Idea too large', it notes that unmanaged ambition without a complete workflow can lead to rework and wasted time. At the bottom, the same text input field and submit button are visible.

11. End-to-End Workflow

Stage	Operation
Input	Upload documents and enter a question.
Extraction	Select parser based on PDF/DOCX/TXT/PPTX extension.
Cleaning	Normalize whitespace and punctuation spacing.
Chunking	Create overlapping text chunks.
Embedding	Convert chunks into vectors.
Retrieval	Compare question and chunk vectors with cosine similarity.
Prompting	Build grounded context with source metadata.
Generation	Mistral generates the response.
Output	Streamlit displays the answer and sources.

12. Real-World Applications

- Academic research: search and compare research papers and generate summaries.
- Corporate knowledge bases: answer questions from internal reports, policies, manuals, and documentation.
- Education: allow students to question textbooks, notes, and study material.
- Technical documentation: help engineers locate procedures and specifications.
- Research and development: compare methods, datasets, findings, and conclusions.
- Legal document review: locate relevant clauses across large document sets, subject to professional validation.

13. Advantages

- Reduces manual searching through long documents.
- Supports multiple document formats.
- Semantic retrieval is based on meaning rather than exact wording.
- Source metadata helps trace retrieved information.
- The prompt is designed to reduce unsupported answers.
- Multiple documents can be searched together.
- The Streamlit UI provides an accessible demonstration interface.

14. Limitations

- The supplied PDF extraction uses `page.get_text()`, so scanned/image-only PDFs may require OCR.
- Embedding and LLM calls require valid API access and network connectivity.
- Only the top five retrieved chunks are passed to the answer model.
- Large-scale deployment would benefit from a persistent vector database.
- Summarization is limited by the configured maximum input word count.
- Production deployment would require stronger security, privacy, authentication, monitoring, and evaluation.

15. Future Scope

- Add OCR for scanned PDFs.
- Add a persistent vector database.
- Combine semantic and keyword retrieval.
- Add dedicated document-comparison views.
- Provide clickable citations to exact source pages.
- Add retrieval and answer-quality evaluation.
- Add authentication and access control for private documents.

16. Conclusion

The AI Research Assistant combines document processing, chunking, embeddings, semantic retrieval, prompt engineering, and LLM generation into one application.

Its main purpose is to make large research document collections easier to search and understand while grounding generated answers in retrieved source content.

The architecture is suitable for a hackathon prototype and can be extended into a production document-intelligence system with OCR, scalable vector storage, stronger evaluation, and security controls.

17. Source Note

The technical description in this report is based on the supplied project implementation, including the Dataprocessing.py document-processing module and the backend and Streamlit frontend code provided for the AI Research Assistant.